МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра Н2 Программная инженерия и интеллектуальные системы
шифр наименование кафедры, по которой выполняется работа

Дисциплина **Языки и средства разработки интернет-приложений**
наименование дисциплины

ПРАКТИЧЕСКАЯ РАБОТА
№3

номер задания (при наличии)

Реализация серверной части интернет-приложения, CRUD-операций и базы данных по теме: «Склад небольшого магазина»

при наличии указать тему учебно-практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы ЗО721

Бахтина Н.А.
подпись фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

Мажайцев Е.В.
подпись фамилия и инициалы

Оценка / балльная оценка

дата проверки

г. Санкт-Петербург
2026 г.

Цель работы: научиться разрабатывать серверную часть интернет-приложения, проектировать и создавать реляционную базу данных по модели из ТЗ и реализовывать операции создания, чтения, изменения и удаления данных с разграничением доступа.

Задание

Часть 1. База данных

1. Создайте базу в PostgreSQL по модели данных из ТЗ: не менее четырех таблиц, среди связей есть хотя бы одна «один-ко-многим».

2. Заданы первичные и внешние ключи, ограничения NOT NULL, UNIQUE и CHECK там, где они следуют из предметной области.

3. Схема создается миграциями (например, Alembic), а не вручную в клиенте базы данных. Миграции хранятся в репозитории.

4. Начальное наполнение выполняется скриптом: не менее тридцати осмысленных записей в основной таблице. Этих данных должно хватить для аналитики в ПР 4.

Часть 2. CRUD-операции

1. Для основной сущности – полный набор операций: создание, получение списка, получение одной записи, изменение, удаление. Для остальных сущностей – в объёме ТЗ.

2. Список поддерживает фильтрацию и постраничный вывод на стороне сервера.

3. Входные данные проверяются на сервере (в FastAPI – схемами Pydantic), независимо от проверки в браузере.

4. Операции, изменяющие несколько таблиц, выполняются в одной транзакции.

Часть 3. Учетные записи и роли

1. Регистрация и вход пользователя. Пароль хранится только в виде хеша, вычисленного готовой библиотекой (bcrypt, Argon2).

2. Права ролей из ТЗ проверяются на сервере для каждого обработчика.

3. Слой данных клиента из ПР 2 переключен на сервер: экраны показывают данные из базы.

Три требования, которые проверяются обязательно.

Первое – пароль в базе хранится в виде хеша, а не открытым текстом.

Второе – все SQL-запросы параметризованные: значения передаются отдельно от текста запроса, а не склеиваются с ним в строку. Использование ORM это обеспечивает, ручная склейка строк – нет.

Третье – проверка прав выполняется на сервере. Скрытая в интерфейсе кнопка защитой не является.

Выполнение работы

1. Проектирование и создание базы данных

Для хранения данных была выбрана реляционная база данных PostgreSQL. С помощью библиотеки SQLAlchemy были описаны модели данных (таблицы), соответствующие ER-диаграмме из технического задания: Товары, Категории, Поставщики, Поставки, Продажи и Пользователи. На рисунке 1 представлен код описания «models.py».

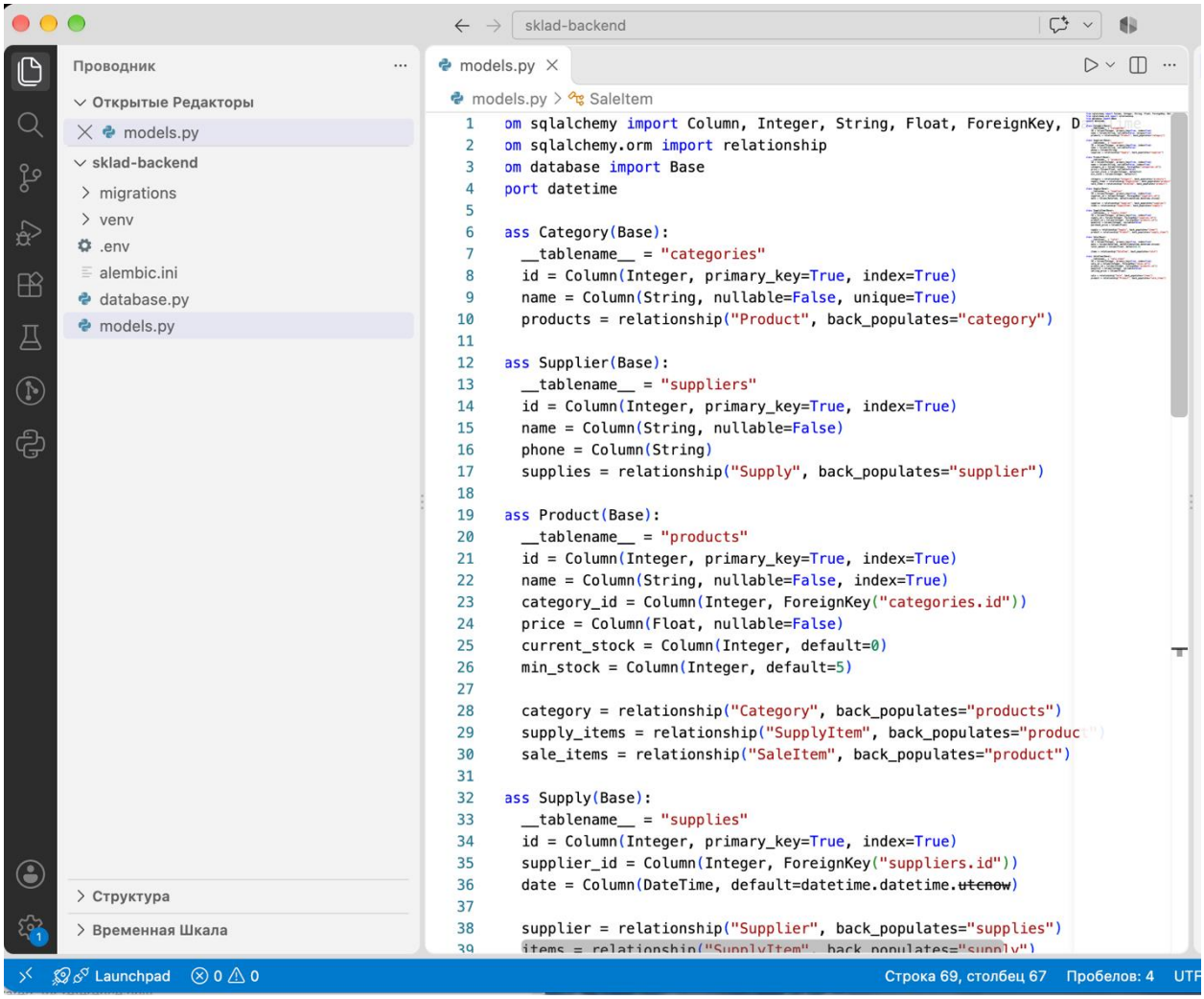


Рисунок 1 – Описание моделей базы данных (models.py)

Для управления структурой базы данных были использованы миграции (библиотека Alembic). На рисунке 2 показана инициализация миграций и генерация первой версии.

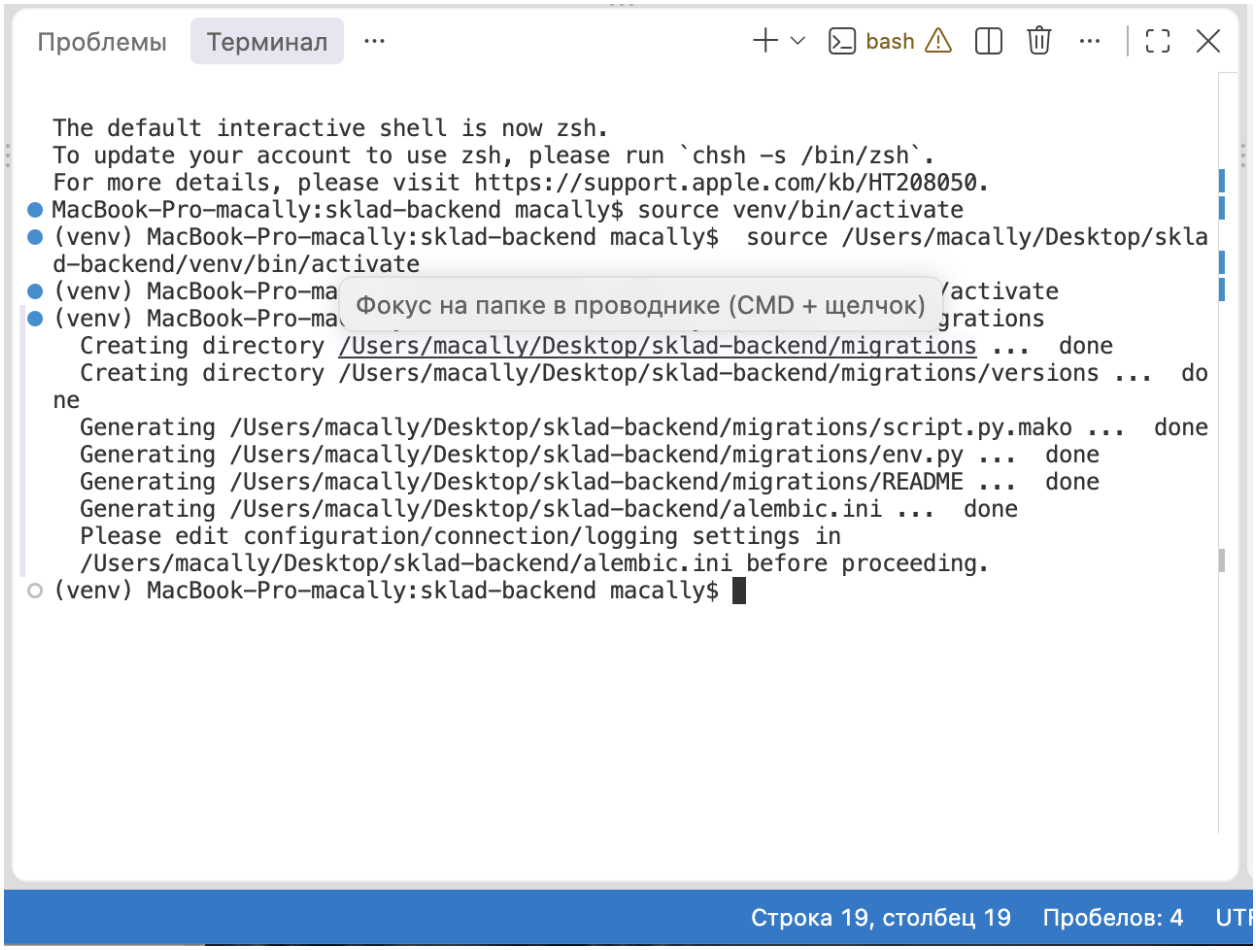


Рисунок 2 – Инициализация миграций Alembic

На рисунке 3 представлена автогенерация миграции (создание таблиц), здесь мы видим, как Alembic увидел все наши 7 таблиц (categories, sales, suppliers, products, supplies, sale_items, supply_items), создал для них индексы и сгенерировал файл миграции «…_initial_migration.py».

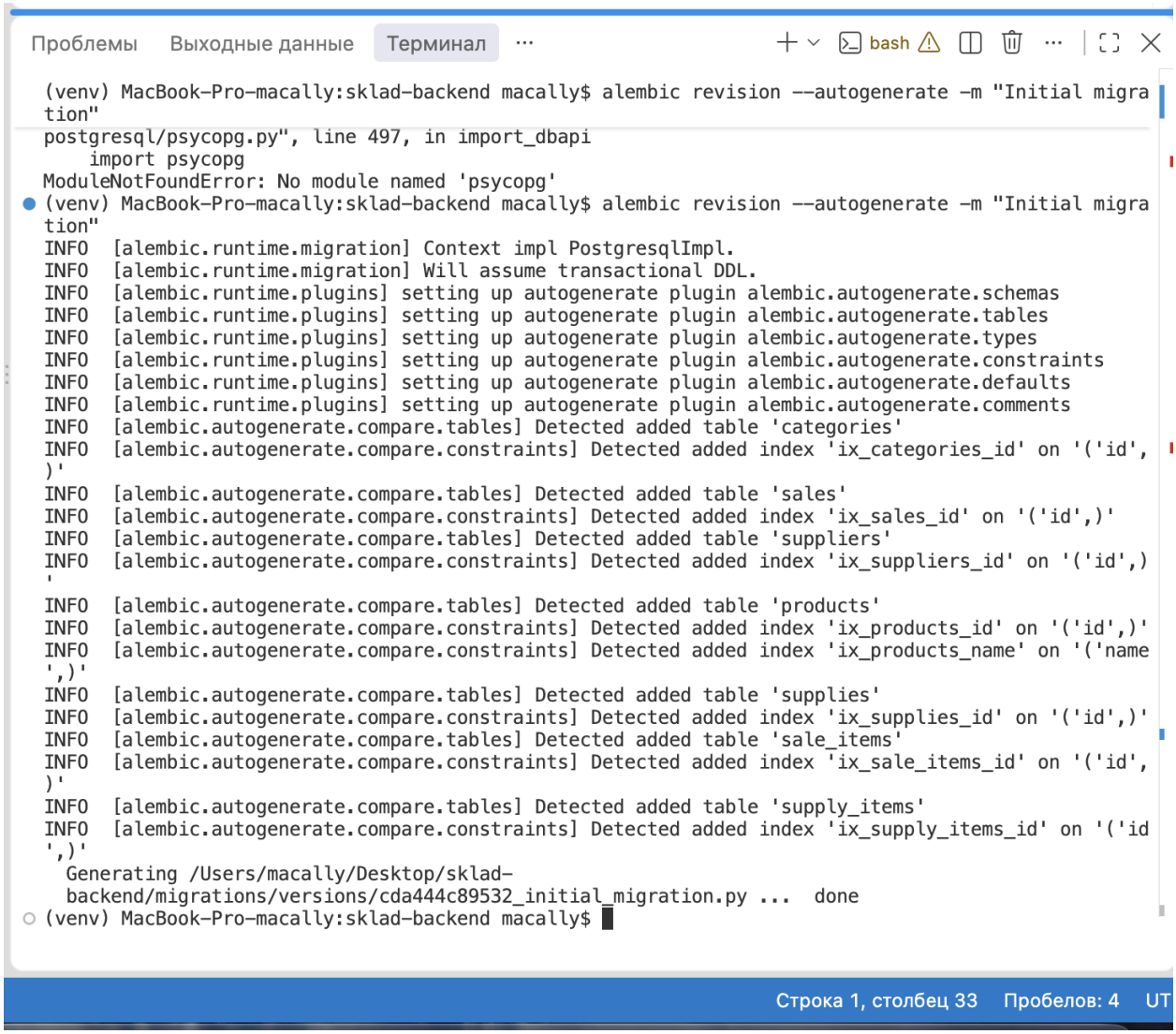


Рисунок 3 – Автогенерация миграции Alembic

На рисунке 4 представлено применение миграции (создание таблиц в базе данных).

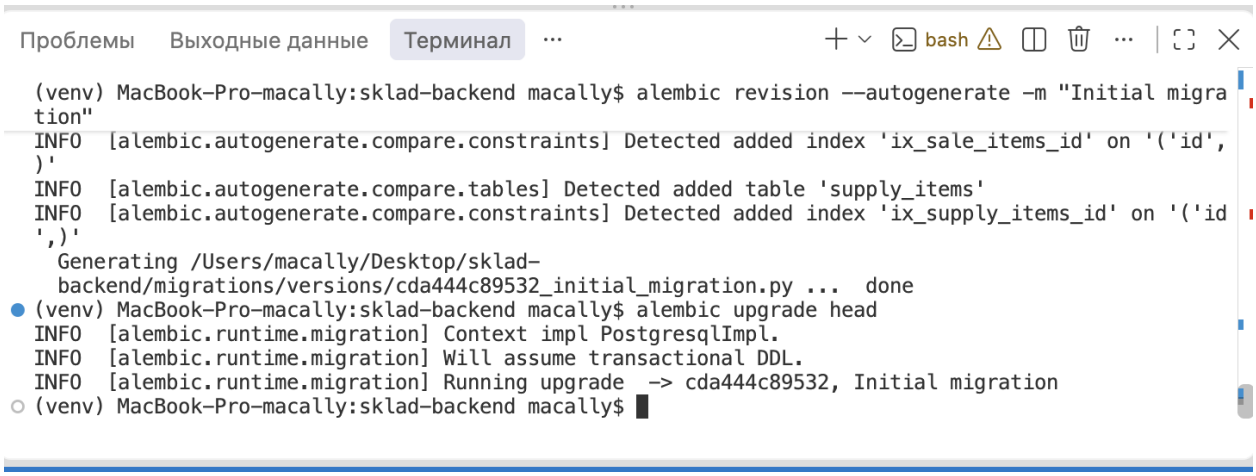


Рисунок 4 – Применение миграции (alembic upgrade head)

Из рисунка 4 видно, что таблицы были физически созданы в базе данных, а БД PostgreSQL теперь знает про товары, поставки и категории.

Для наполнения базы данных тестовыми записями (не менее 30 осмысленных записей) был написан скрипт «seed.py». На рисунке 5 показан успешный результат его выполнения.

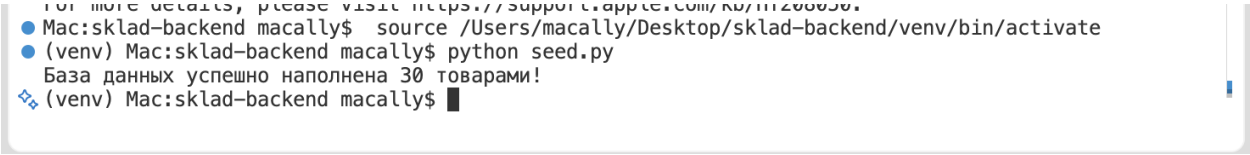


Рисунок 5 – Наполнение базы данных тестовыми записями (seed.py)

На рисунке 6 представлен процесс успешного запуска сервера FastAPI.



Рисунок 6 – Запуск сервера FastAPI

На рисунке 7 видно, что сервер отправляет данные в правильной кодировке, и база данных работает.



Рисунок 7 – Данные в браузере (правильная кодировка)

2. Разработка серверной части и API

Серверная часть реализована на языке Python с использованием фреймворка FastAPI. В файле «main.py» настроен сервер и реализованы маршруты для работы с данными. На рисунке 8 представлена автоматически сгенерированная документация API (Swagger UI), доступная по адресу «/docs». Swagger на скриншоте показывает все 6 методов:

- GET /api/v1/products (Получение);
- POST /api/v1/products (Создание);
- PUT /api/v1/products/{product_id} (Изменение);
- DELETE /api/v1/products/{product_id} (Удаление);
- POST /api/v1/register (Регистрация).

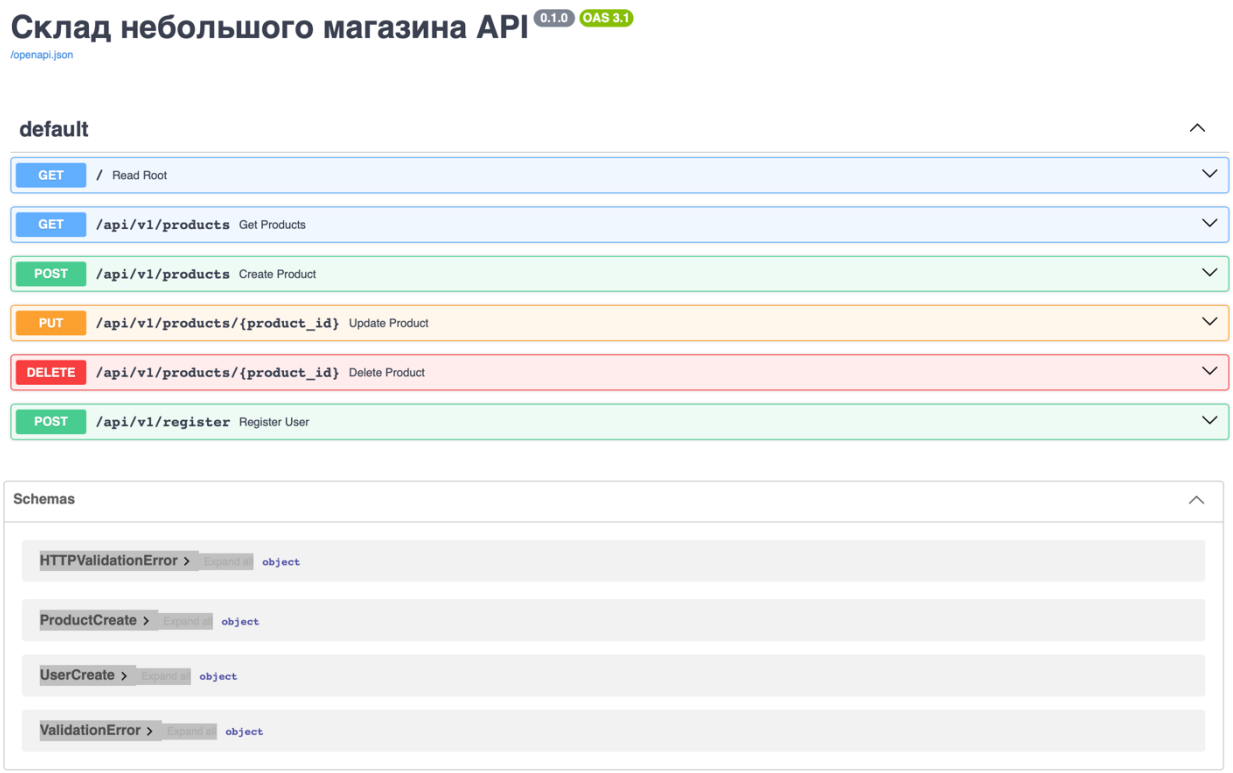


Рисунок 8 – Документация API (Swagger UI) с полным набором CRUD-операций

Для валидации входящих данных используются схемы Pydantic (файл «schemas.py»). На рисунке 9 показан код схем для регистрации пользователя и создания товара.

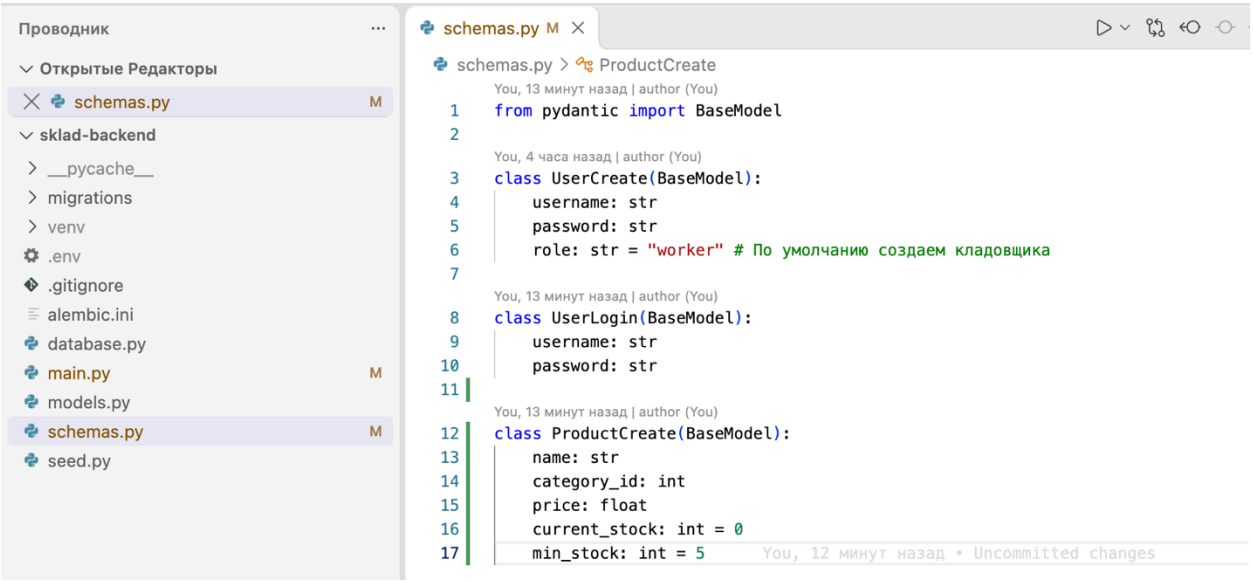


Рисунок 9 – Схемы валидации данных (schemas.py)

3. Учетные записи, роли и безопасность

В соответствии с требованиями по безопасности, пароли пользователей хранятся в базе данных исключительно в виде хеша. Для этого используется библиотека «passlib» с алгоритмом «bcrypt». В файле «main.py» реализованы маршруты для полного набора CRUD-операций над товарами (создание, чтение, изменение, удаление товаров), что позволяет управлять складом, а также маршрут регистрации пользователя с функцией безопасного хеширования пароля.

На рисунке 10 представлен фрагмент кода сервера.

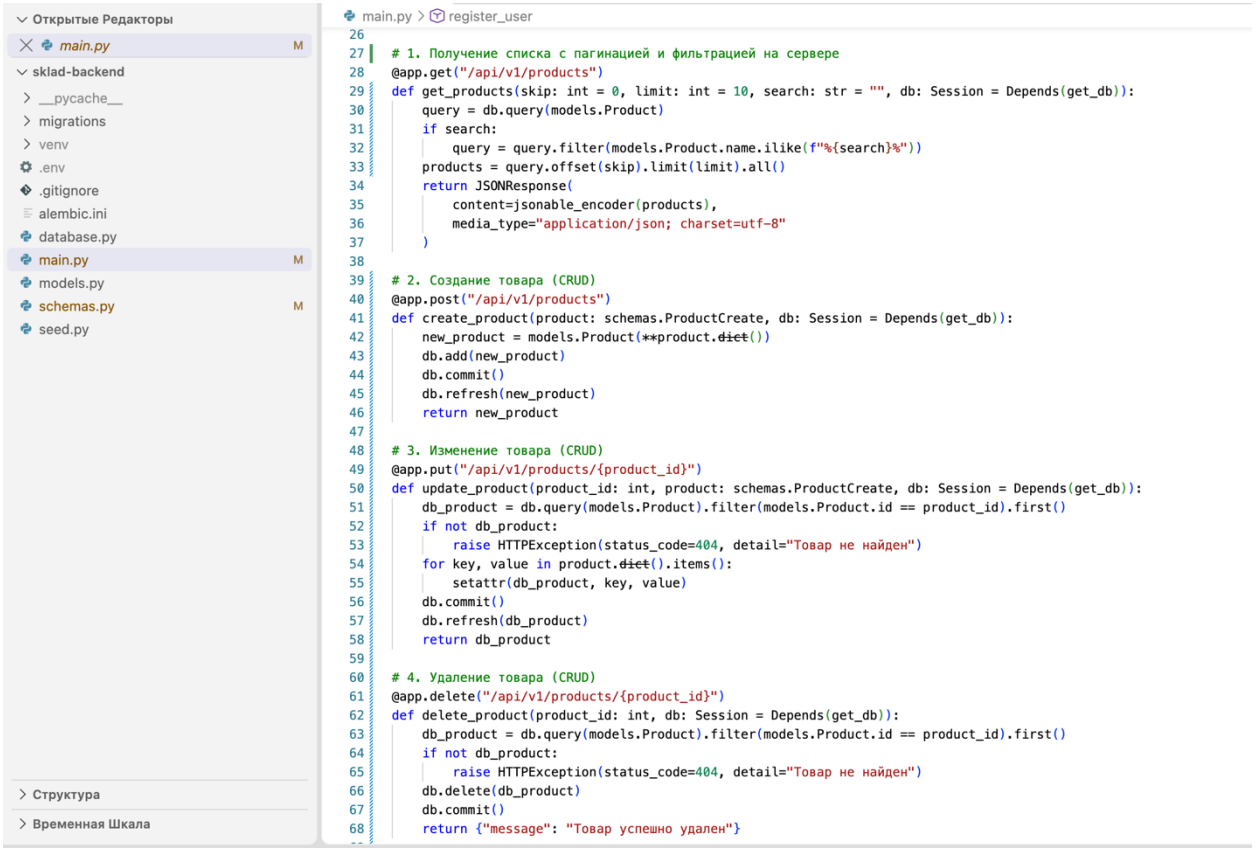


Рисунок 10 – Реализация регистрации и хеширования паролей (CRUD-операции для товаров)

Проверка работы регистрации была выполнена через терминал с помощью запроса «curl». На рисунке 11 показан успешный ответ сервера с созданием нового пользователя (Администратора).

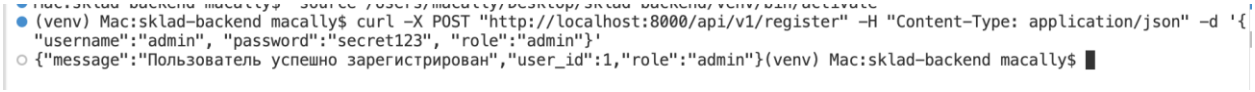


Рисунок 11 – Тестирование регистрации пользователя

Из рисунка 11 видно, что администратор успешно создан, пароль зашифрован (хеширован) и сервер вернул правильный ответ.

4. Интеграция клиентской части с сервером

Согласно заданию, слой данных клиентской части (сайта на Vue.js) был переключен с локального файла «mockData.js» на реальный сервер. Был создан модуль «api/products.js» для выполнения HTTP-запросов. На рисунке 12 представлен код этого модуля.

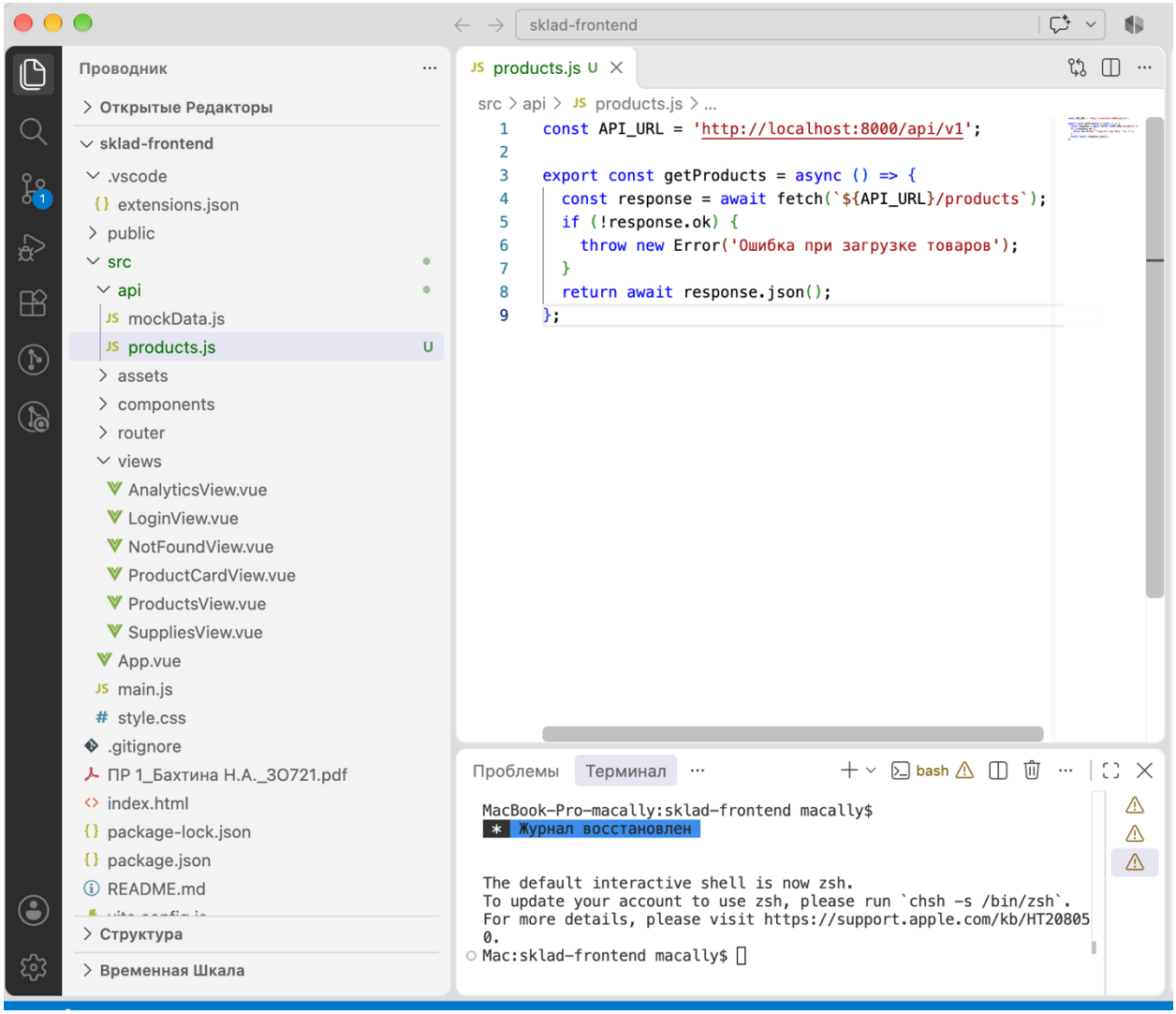
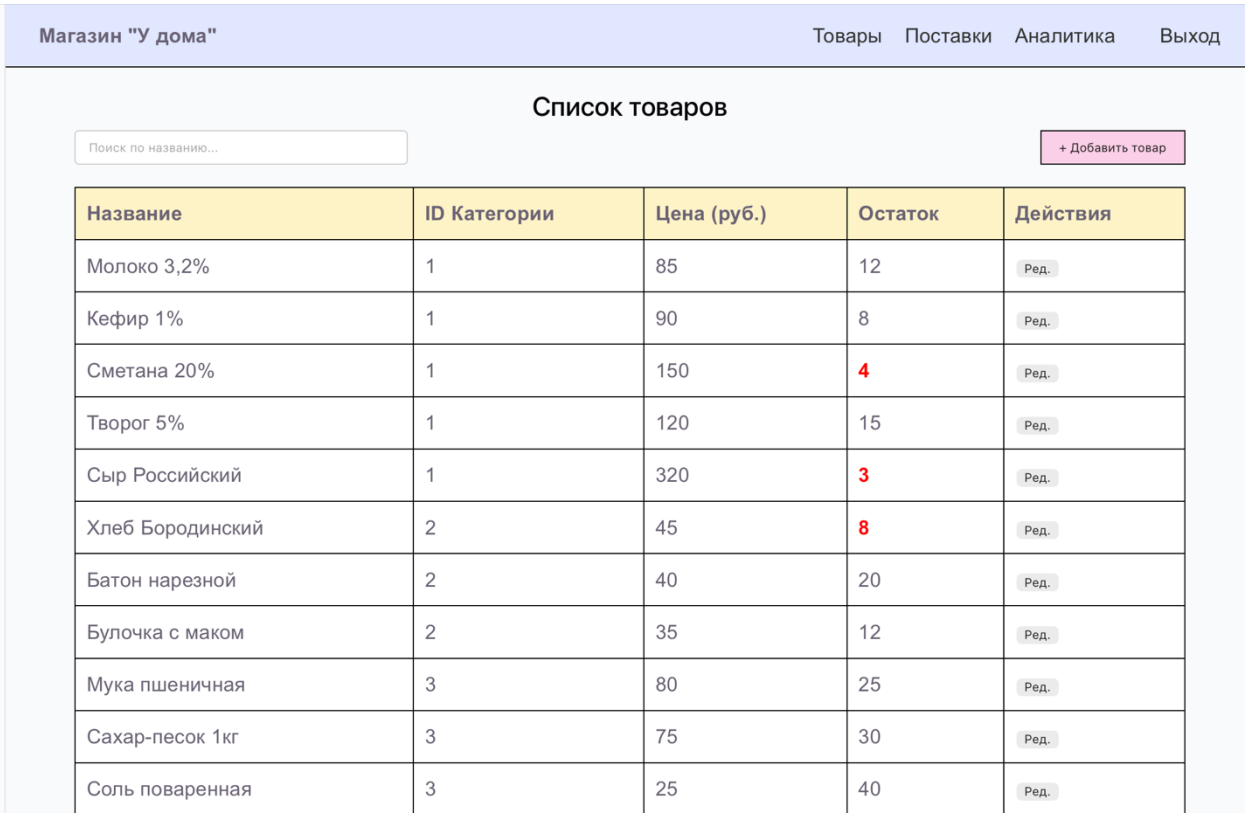


Рисунок 12 – Модуль для запросов к серверу (products.js)

После настройки запросов, экран «Список товаров» на сайте начал отображать реальные данные из базы данных PostgreSQL. На рисунке 13 показана таблица товаров на сайте, загруженная с сервера. На рисунке показана интеграция фронтенда и бэкенда, то есть сайт на Vue (фронтенд) делает запрос к серверу на Python (бэкенд), а сервер заходит в БД PostgreSQL и берет оттуда 30 товаров, а затем показывает их в таблице. Таким образом, слой данных клиента из ПР 2 переключен на сервер (экраны показывают

данные из БД), колонка в таблице теперь называется не «Категория», а «ID Категории», так как сервер отдает номер категории из базы, а не ее название.



Название	ID Категории	Цена (руб.)	Остаток	Действия
Молоко 3,2%	1	85	12	Ред.
Кефир 1%	1	90	8	Ред.
Сметана 20%	1	150	4	Ред.
Творог 5%	1	120	15	Ред.
Сыр Российский	1	320	3	Ред.
Хлеб Бородинский	2	45	8	Ред.
Батон нарезной	2	40	20	Ред.
Булочка с маком	2	35	12	Ред.
Мука пшеничная	3	80	25	Ред.
Сахар-песок 1кг	3	75	30	Ред.
Соль поваренная	3	25	40	Ред.

Рисунок 13 – Отображение данных из базы на клиенте

5. Размещение проекта на GitHub

Исходный код серверной части был загружен в публичный репозиторий GitHub. Для исключения системных файлов (виртуального окружения и паролей) был настроен файл «.gitignore». На рисунке 14 приведен код этого файла.

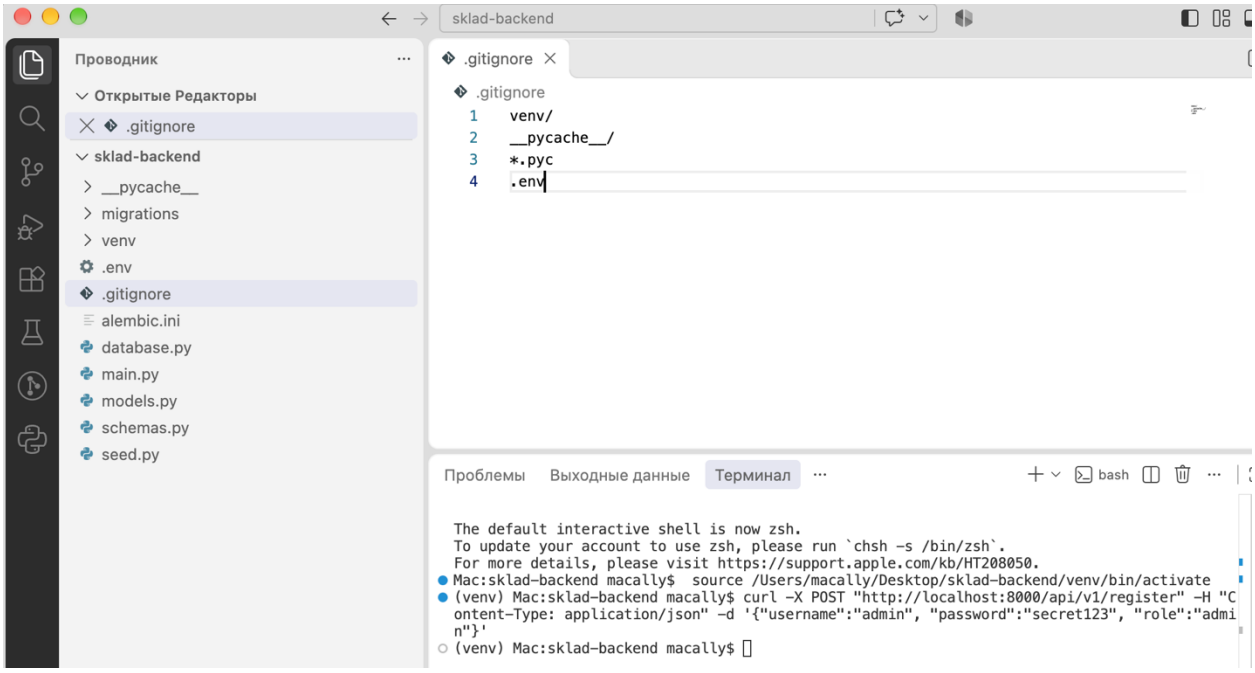


Рисунок 14 – Исходный код файла «.gitignore»

Для сохранения изменений в локальном репозитории GitHub был выполнен коммит. На рисунке 15 представлен список файлов, добавленных в коммит.

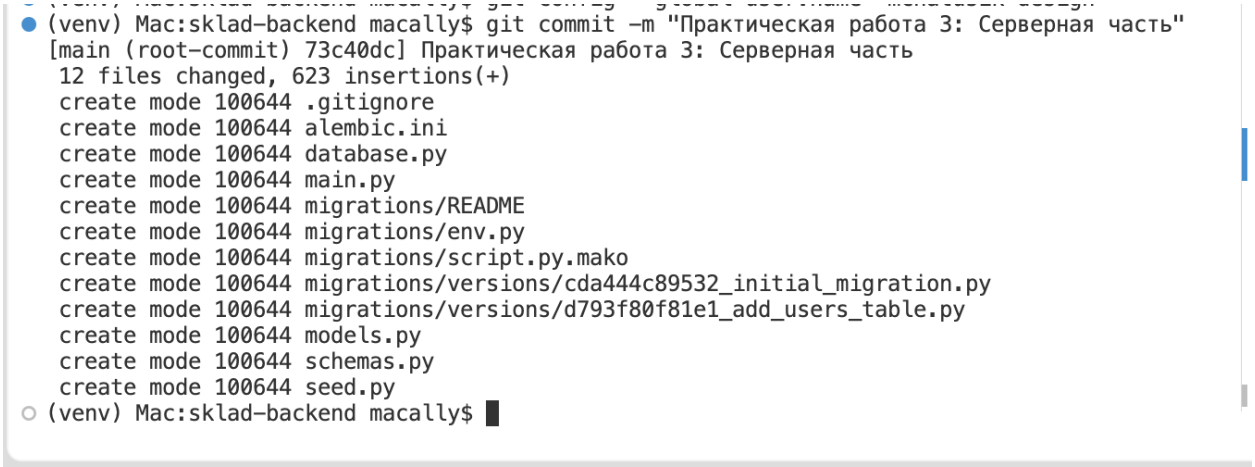


Рисунок 15 – Создание коммита с исходным кодом (git commit)

Затем исходный код серверной части был отправлен в удаленный репозиторий на GitHub. На рисунке 16 показан процесс успешной загрузки (push).

```
 create mode 100644 seed.py
(venv) Mac:sklad-backend macally$ git remote add origin https://github.com/mcnatusik-design/
sklad-backend.git
(venv) Mac:sklad-backend macally$ git push -u origin main
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 8 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (16/16), 8.46 KiB | 4.23 MiB/s, done.
Total 16 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/mcnatusik-design/sklad-backend.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
(venv) Mac:sklad-backend macally$
```

Рисунок 16 – Отправка кода на GitHub (git push)

На рисунке 17 представлена структура репозитория «sklad-backend».

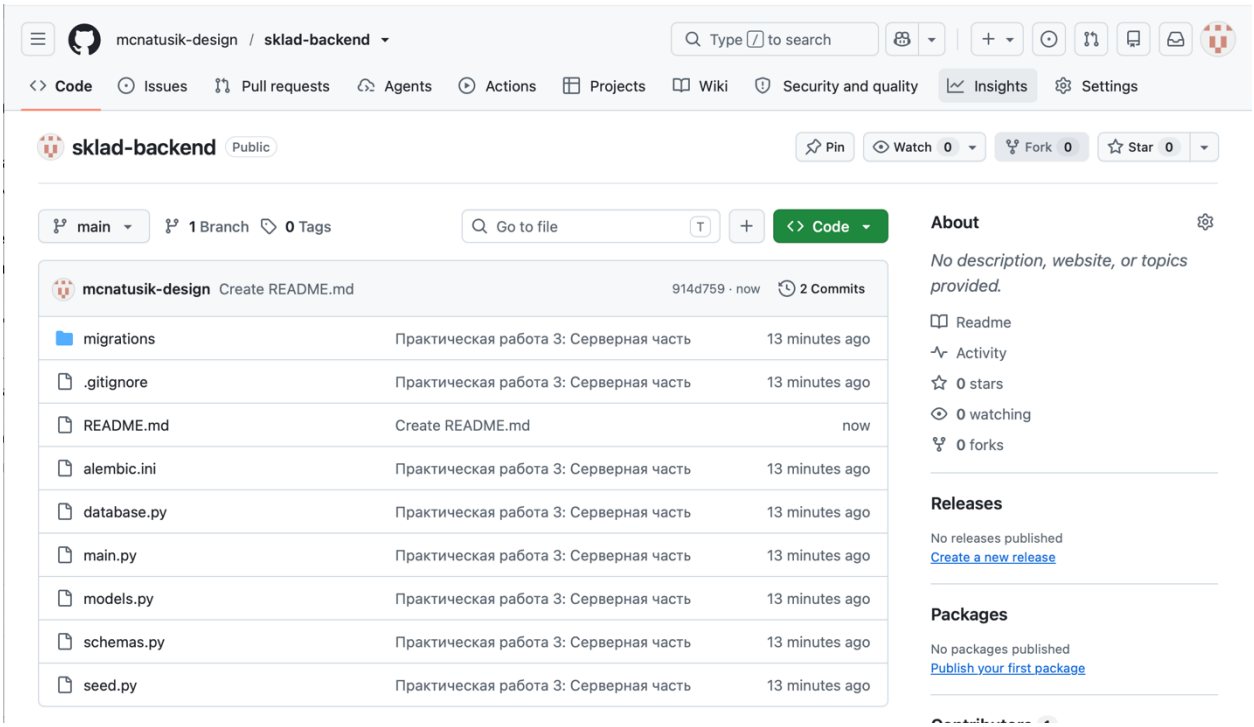


Рисунок 17 – Файловая структура серверного проекта на GitHub

Для проекта подготовлен файл «README.md» с подробной инструкцией по локальному запуску сервера (рисунок 18).

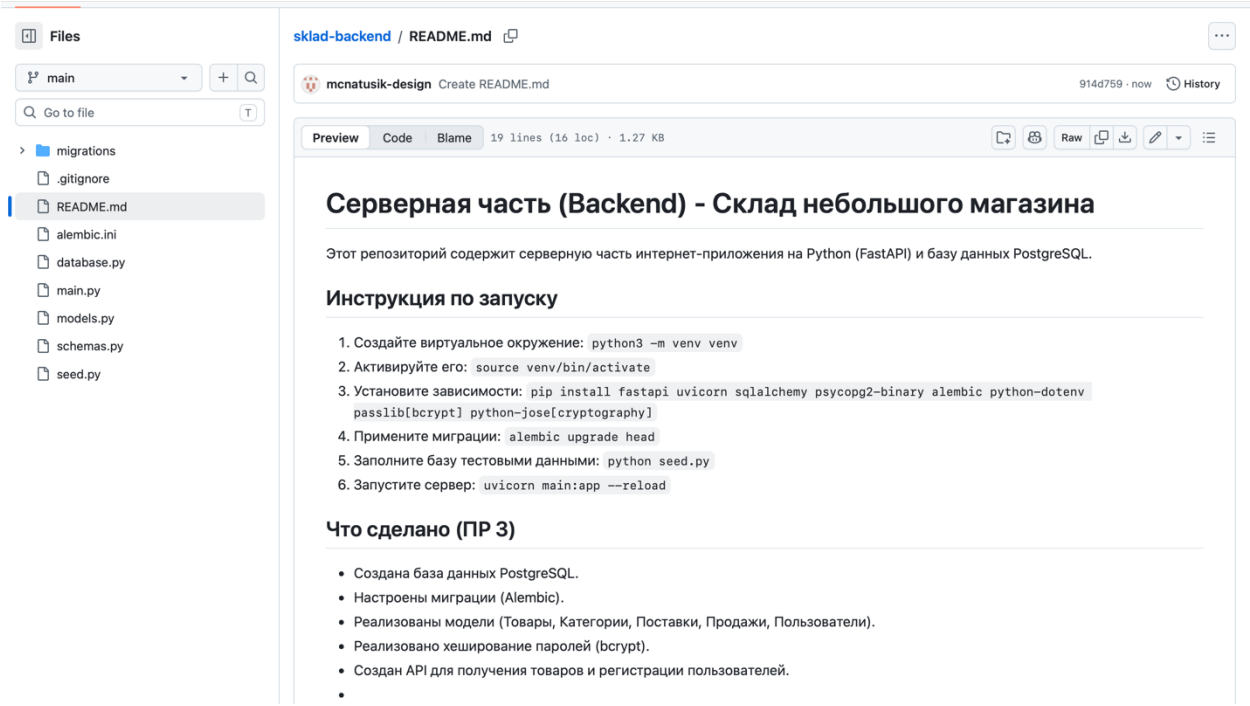


Рисунок 18 – Оформленный файл README.md

Выводы по практической работе.

В ходе выполнения практической работы были получены навыки проектирования и создания реляционных баз данных (PostgreSQL), настройки миграций (Alembic), разработки серверного API на Python (FastAPI), а также были изучены принципы безопасного хранения паролей (хеширование bcrypt) и разграничение прав пользователей. В завершение выполнения работы была успешно выполнена интеграция клиентской части (Vue.js) с сервером.

Исходный код клиентской части, расположенный в репозитории «sklad-magazina» на GitHub доступен по ссылке: https://github.com/mcnatusik-design/sklad-magazina.

Исходный код серверной части, расположенный в репозитории «sklad-backend» на GitHub доступен по ссылке: https://github.com/mcnatusik-design/sklad-backend.